

Shokunin Technical Overview

v4.2.2

github.com/EliasOulkadi/shokunin

62 Skills | Windows + Linux | Open Source MIT

Technical Overview

Open Source -- MIT License

github.com/EliasOulkadi/shokunin



1. Architecture Overview

Shokunin is a production-grade AI coding ecosystem that provides persistent memory via ChromaDB, 62 specialized AI skills across 10 domains, MCP (Model Context Protocol) servers for tool integration, and a declarative self-update system. It runs fully offline with zero server infrastructure and zero API costs.

The ecosystem operates on two levels: a local runtime (`~/shokunin/`) that manages memory, sessions, and script execution, and a distribution layer (`.pack/`) that contains 62 skills, MCP server implementations, CLI tools, and installer scripts. The update system uses a declarative JSON manifest (`shokunin.json`) with template-based regeneration and backup/rollback.

2. Memory System Architecture

The memory system stores AI agent context across sessions using three complementary layers:

- Layer 1 -- ChromaDB Vector Store: Semantic embeddings enable similarity search across all stored entries. Default embedding model is all-MiniLM-L6-v2 (384 dimensions), running locally via ONNX with no API dependency. Collection stores documents with metadata including type, tags, project, session_id, and timestamp.

- Layer 2 -- JSONL Transaction Log: Every interaction is logged as line-delimited JSON in per-session files. Provides structured, machine-parseable records for programmatic analysis and replay. Includes entry type, timestamp, session_id, content, and role tracking.
- Layer 3 -- Markdown Session Files: Human-readable summaries for manual review and debugging. Each session produces a .md file with timestamped entries organized by type (decisions, files, commands, checkpoints, session_end).

The MCP server (mcp-server.py, 559 lines) implements JSON-RPC 2.0 over stdin/stdout, requiring zero network configuration. It exposes 9 tools for memory operations, session management, and file path verification. All Python code is 100% type-annotated with lazy ChromaDB initialization and double-checked locking for thread safety.

3. Multi-Strategy Search

The multi_search_context tool combines four search strategies with Reciprocal Rank Fusion:

- Vector Search: ChromaDB semantic similarity using cosine distance on document embeddings.
- BM25 Keyword Search: Term frequency-inverse document frequency with tunable k1 and b parameters. Builds an index from session documents and tokenizes for full-text matching.
- Temporal Filtering: Date-range filtering using ISO 8601 timestamps (from_date/to_date) to narrow results to specific time windows.
- Reciprocal Rank Fusion (RRF): Merges vector and BM25 result lists with a weighted k constant (default 60). Entries appearing in both lists receive combined scores. Deduplication by session_id + content hash prevents double-weighting.

4. Freshness Decay System

One of the hardest problems in persistent memory is retrieval shaping: ensuring recent context surfaces before stale claims. Shokunin solves this with exponential freshness decay blended with vector similarity.

Each memory entry carries a UTC timestamp. When searching with freshness_boost > 0, the similarity score becomes a weighted blend:

```
blended_score = (1 - freshness_boost) * vector_similarity
                + freshness_boost * exp(-days_since_stored / 30)
```

At freshness_boost = 0.0, results are purely relevance-based (default). At 0.5, relevance and recency receive equal weight. At 1.0, results are purely recency-based. The 30-day half-life means an entry from 30 days ago receives 50% of the recency weight of an entry stored today. After 90 days, recency weight drops below 5%.

This system prevents the common failure mode where an agent recalls 'the auth helper lives at src/x.ts' from a memory written two weeks ago, after the file has been refactored. The freshness decay naturally deprioritizes stale claims, while the verify_file_path tool (detailed below) provides explicit verification.

5. Claim Verification System

The verify_file_path MCP tool is the centerpiece of Shokunin's approach to memory safety. When an agent

recalls a specific file path from old memory, the tool validates whether the path still exists on the local filesystem before the agent acts on it.

The verification workflow:

- 1. Agent recalls a memory mentioning a file path (e.g., 'auth helper at src/auth.ts')
- 2. Agent calls `verify_file_path('src/auth.ts')` before acting
- 3. If the file exists: returns {exists: true, path, last_modified, kind}
- 4. If the file does not exist: returns {exists: false}. Agent searches memory for newer entries describing the refactoring, then greps the codebase for the current location
- 5. Agent acts only after confirming the path, never on stale claims alone

The CLAUDE.md and AGENTS.md files contain a mandatory CLAIM VERIFICATION section that instructs the agent: 'When a memory mentions a specific FILE PATH, FUNCTION NAME, or CONFIG FLAG: It is a CLAIM FROM A FROZEN POINT IN TIME -- not guaranteed current. BEFORE acting on it, verify the file/function/flag still exists.'

Claim types (`claim_file`, `claim_function`, `claim_flag`, `claim_api`) extend `VALID_TYPES` to enable structured fact tracking. Each claim carries enriched metadata including the claimed path, function name, or flag value, plus a `verified_at` timestamp when the claim is validated.

6. MCP Tool Suite (9 Tools)

All tools are exposed via the MCP server over JSON-RPC 2.0 stdin/stdout transport:

- `store_context` -- Store text in persistent memory with type (12 valid types including decision, file, command, preference, checkpoint, session_end, claim_file, claim_function, claim_flag, claim_api), tags, project, and session_id
- `search_context` -- Vector similarity search with `freshness_boost` parameter (0.0-1.0). Returns ranked results with similarity scores, metadata, and timestamps
- `get_session_summary` -- Retrieve all entries for a specific session with aggregated statistics (tags, projects, types). Limited to 100 entries maximum to prevent OOM
- `multi_search_context` -- Combined vector + BM25 + temporal filter with RRF fusion. Accepts `freshness_boost` and date range parameters
- `consolidate_memories` -- Summarize old entries per project by extracting common terms and generating consolidated summary documents
- `list_sessions` -- Recent sessions with metadata (project, entry count, first/last timestamp, types). Filters out healthcheck and test entries
- `continue_session` -- Full context load for session resume. Returns decisions, files, commands, checkpoints, and JSONL message transcript
- `save_message` -- Record an individual message exchange (user or assistant) in the session transcript as line-delimited JSON
- `verify_file_path` -- Validate file/directory existence on local filesystem. Supports ~ expansion and relative paths. Returns exists, path, last_modified, and kind

7. Skills Ecosystem -- 62 Skills

Every skill includes YAML frontmatter with name and description for automatic trigger matching. All 62 skills

include four mandatory sections: Workflow (numbered procedural steps), Error Handling (5+ Cause|Fix scenarios), Sources (5+ verifiable references), and Anti-Patterns (5+ Pattern|Problem|Fix entries). Average skill length is 257 lines.

Infrastructure (5)

- docker -- Multi-stage Dockerfiles with BuildKit, distroless, seccomp profiles, and CVE scanning
- kubernetes

Backend (5)

- api-forge -- REST/GraphQL APIs with OpenAPI 3.1, rate limiting, webhooks, idempotency
- auth-arch

Frontend (11)

- component-forge -- React/Vue/Svelte components with 5 states (loading/empty/error/success/idle)
- responsive

Mobile (2)

- flutter -- Clean Architecture, Riverpod, GoRouter, Impeller, Pigeon
- react-native

Quality (7)

- test-commander -- Testing Trophy (80% integration), MSW, Playwright, visual regression
- performance

Content & Business (6)

communication, content-marketing, business-proposals, seo-geo, translate-craft, documentation

Productivity (8)

git-workflow, windows-powershell, runbook-gen, strategy, design, finance, legal-counsel, whendone-plus

Documents (3)

kami, kagen, portfolio-auto

AI Agents & Ecosystem (9)

agent-browser, agent-tools, skill-creator, shokunin-update, chromadb, memory

Extras & Language (7)

playwright, web-security, plan, find-skills, efficient-coding, senior-engineer, humanize, research, init

8. Quality and Testing Infrastructure

- 29 integration tests covering security (path traversal sanitization with 21 test cases), MCP protocol compliance (tools/list, invalid methods, tool not found), and ChromaDB operations (save, search, recall, session management). All tests pass on every commit.
- CI/CD pipeline with 3 workflows: CI (frontmatter validation, mypy type checking, bandit security scanning), Memory Tests (lint, integration tests, cross-platform validation on Windows and Linux), and Release (automatic versioning and deployment).
- 12 PowerShell scripts with Set-StrictMode -Version Latest, ErrorActionPreference, CmdletBinding, and comment-based help on all public scripts.
- 100% type-annotated Python codebase (from `__future__` import annotations). Lazy ChromaDB initialization with `threading.Lock` for thread safety. `_dispatch` pattern for JSON-RPC routing with proper notification suppression.
- Dependabot for automated dependency updates, CODEOWNERS for review assignment, `.editorconfig` for consistent formatting across editors.

9. Security Architecture

- Path traversal prevention via regex-based session ID sanitization (`_sanitize_id` / `_safe_id`). All special characters replaced with safe alternatives before file I/O.
- Error message sanitization: MCP server returns generic 'Internal server error' to clients; full error details logged server-side only.
- All exception handlers include logging with context (session ID, operation, data involved). No bare except blocks remain in the codebase.
- Double-checked locking pattern on ChromaDB initialization prevents race conditions in multi-threaded access scenarios.
- JSON-RPC 2.0 compliance: notifications (requests without id) receive no response, preventing protocol-level information leaks.

10. Getting Started

Windows one-command install:

```
irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex
```

Linux:

```
bash <(curl -sSL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)
```

Requirements: Windows 10/11 or Linux, Node.js 18+, Python 3.11+, Git. The installer sets up ChromaDB, 62 skills, PowerShell/Linux profiles, MCP servers, scheduled maintenance tasks, and the OpenCode configuration. After installation, open a new terminal and run 'opencode'. The ecosystem activates automatically with persistent memory across sessions.

The skills browser at <https://eliasoulkadi.github.io/shokunin/skills.html> provides an interactive catalog of all 62 skills with search, domain grouping, and expandable content previews.